

Introduction to Message Passing Interface (MPI) 2026-2027

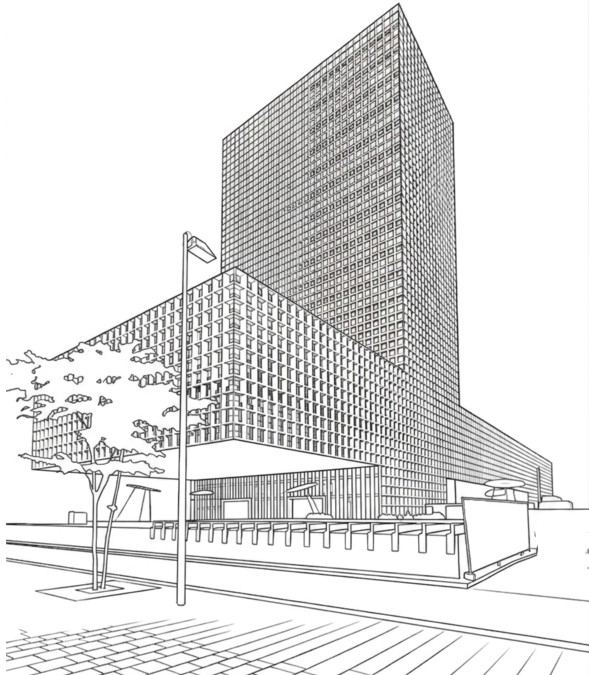
Lesson 1

Matteo Barborini

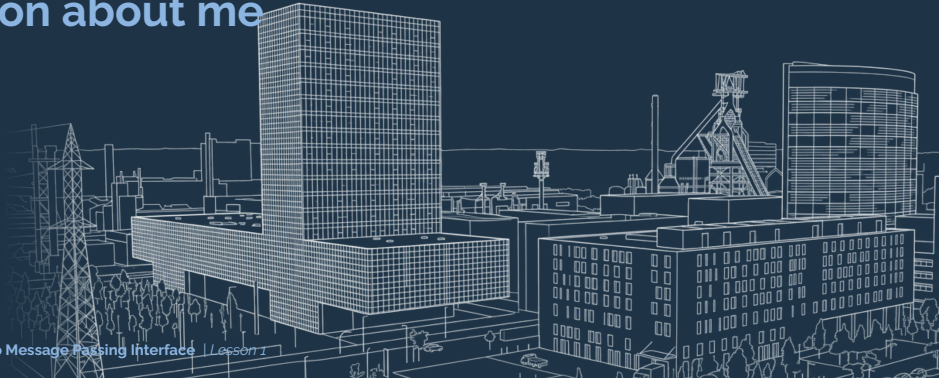
University of Luxembourg - HPC Platform

matteo.barborini@uni.lu

September 15, 2026



Information about me



Title:

Research Scientist at the University of Luxembourg - HPC Platform.

Contacts:

Office: Maison du Nombre, E03 0315010

e-mail: matteo.barborini@uni.lu

Studies:

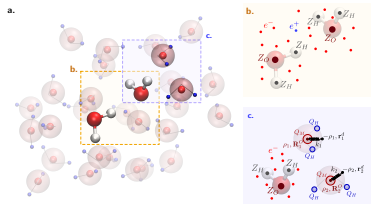
Bachelor and Master degrees in Physics and PhD in Mathemaical modelling.

Research Interests:

Development of Quantum Monte Carlo (QMC) methods¹ for calculations of molecular properties and Materials Science.

¹M. H. Kalos and P. A. Whitlock, *Quantum monte carlo*, (John Wiley & Sons, Ltd, 2008), F. Becca and S. Sorella, *Quantum monte carlo approaches for correlated systems*, (Cambridge University Press, 2017).

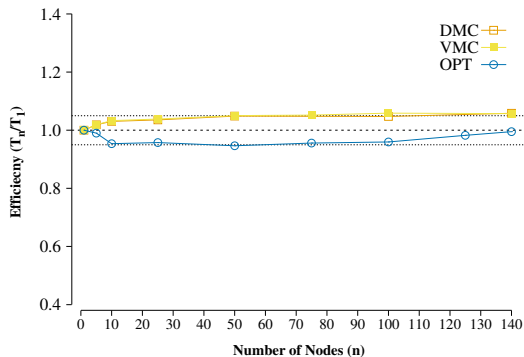
Development of **QMeCha QMC package**^a. With MPI and OpenMP parallelization.



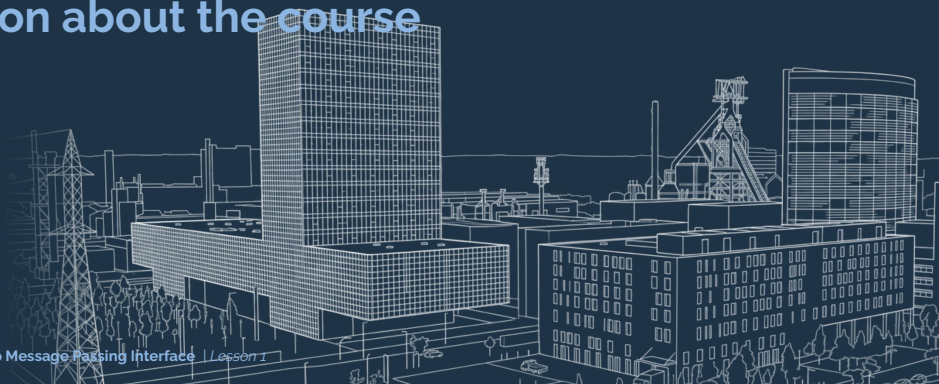
Cluster of molecular systems, to study cohesion energies and structures.

^aM. Barborini et al., "Qmecha: quantum monte carlo package for fermions in embedding environments", *The Journal of Chemical Physics* **164**, 062501 (2026).

Scaling of the calculations as a function of the number of compute nodes. Using 128 cores on each node, with 8 MPI processes with 16 OpenMP threads each.



Information about the course



1. Basic Hardware Concepts in High-Performance Computing

- 1.1 Von Neumann architecture / bottleneck
- 1.2 Hardware Parallelism
- 1.3 HPC system

2. Parallel programming model and basic concepts.

- 2.1 Foster's methodology
- 2.2 Partitioning approaches

3. Speed up and Efficiency

- 3.1 Amdahl's law / Gustafson-Barsis's law
- 3.2 Strong and weak scaling

4. Introduction to the MPI standards

- 4.1 Synchronous and asynchronous communication.
- 4.2 Point-to-point communication.
- 4.3 Collective communications.

Introductory Books:

- **An introduction to parallel programming**, by *Peter Pacheco and Matthew Malensek*
- **Using MPI**, by *William Gropp, Ewing Lusk, and Anthony Skjellum*

Advanced Books:

- **Using Advanced MPI**, by *William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk*
- **Parallel Programming with MPI**, by *Peter Pacheco*

Nice to have books:

- **Programming Your GPU with OpenMP**, by *Tom Deakin and Timothy G. Mattson*

On line references:

- **MPI Forum Documentation** (www.mpi-forum.org/docs/)
- **Open MPI Documentation** (www.open-mpi.org/doc/)

Three tasks each evaluated with a score from 0 to 20:

Task 1 (10% of the final score) 7.10.2026

Task 2 (10% of the final score) 11.11.2026

Task 3 (10% of the final score) 9.12.2026

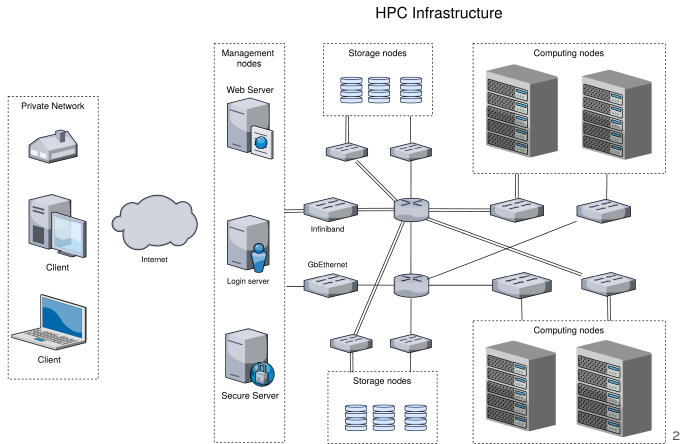
Final Exam oral discussion about the three tasks, implementations, results and possible different approaches to parallelization (70% of total score).

On this last task the student must obtain a score of at least 10/20 on this oral presentation to pass the course.

High Performance Computing (HPC) architecture



Clusters connected by a **grid** network that has a given topology.

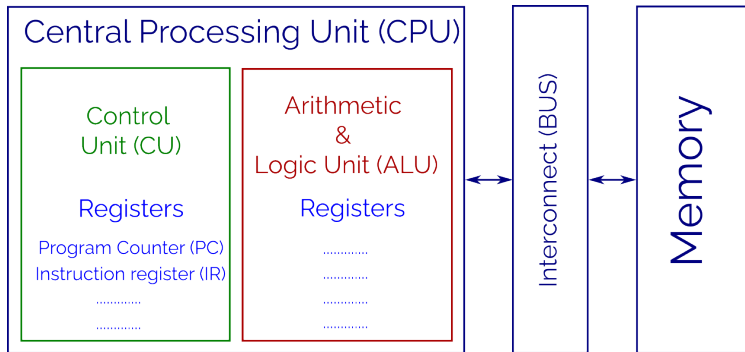


²F. Reghenzani et al., "Timing predictability in high-performance computing with probabilistic real-time", IEEE Access **8**, 208566–208582 (2020).

Architectures of Central processing units



The **Von Neumann Architecture** is an example of **Single Instruction Single Data (SISD)** architecture, where a single processor executes a single instruction stream operating on a single data element in memory.³



The 'classical' architecture based in the Von Neumann model is composed of a **Central Processing Unit (CPU)**, a **Memory Unit** and an **Interconnect (BUS)**.

³M. Godfrey and D. Hendry, "The computer as von neumann planned it", IEEE Annals of the History of Computing **15**, 11–21 (1993).

Central Processing Unit (CPU)

Control
Unit (CU)

Registers

Program Counter (PC)

Instruction register (IR)

.....

.....

Arithmetic
&
Logic Unit (ALU)

Registers

.....

.....

.....

.....

ALU: executes the arithmetic and logical instructions.

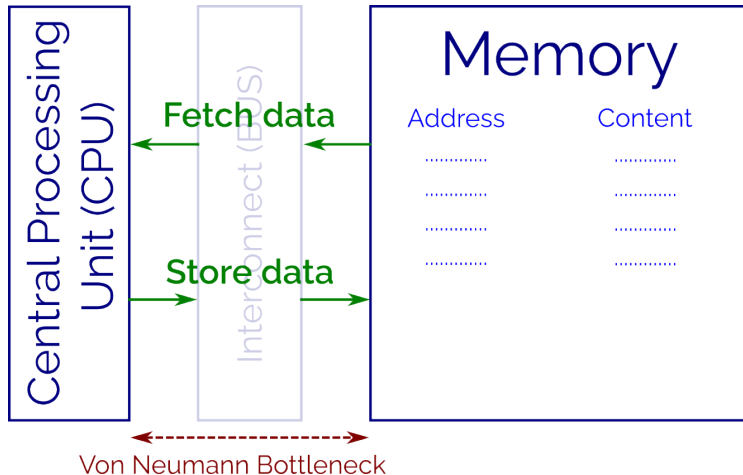
CU: Decides the instructions (and the order of the instructions) that the ALU has to execute.

Registers: are small, fast storage locations that are directly accessible by the CPU. Within them we have the **Program Counter (PC)** that contains the address of the next instruction to be executed and the **Instruction Register (IR)** that contains the instruction currently being executed.

The distance between the CPU and the memory is often called the **Von Neumann bottleneck** since it limits the rate at which instructions can be executed.

(In 2010 CPUs are capable of executing instructions more than one hundred times faster than they can fetch items from main memory.)^a

^aP. S. Pacheco and M. Malensek, "Chapter 2 - parallel hardware and parallel software", in *An introduction to parallel programming (second edition)*, Second Edition (Morgan Kaufmann, Philadelphia, 2022), pp. 17–88.



Cache: collection of memory locations that can be accessed in less time than some other memory locations. L3, L2, L1, the last closer to the CPU. Divided in cache lines/blocks.

Where and how is the memory information stored on the Cache lines/blocks?

- fully associative

- direct mapped

- n-way set associative

Virtual Memory: developed so that main memory can function as a cache for secondary storage. (Usually not exploited in scientific computing)

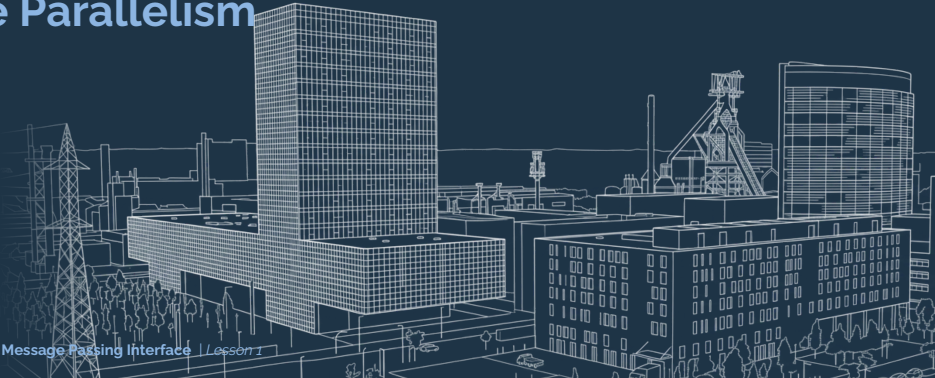
Low-level parallelism:

- Instruction-level parallelism (ILP) (finer-grained parallelism) that can be by *pipelining* or *multiple issue*.

- Thread-level parallelism (TLP), hardware multithreading (coarser-grained parallelism)

Here the CPU must *speculate* on the next instruction to be executed.

Hardware Parallelism



Flynn's Taxonomy⁴ is a classification of computer architectures based on the number of concurrent instruction (single or multiple) and data streams (single or multiple) available in the architecture.

Single Instruction Single Data (SISD) Von Neumann architecture.

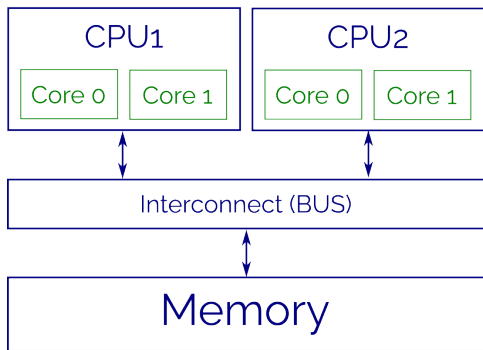
Single Instruction Multiple Data (SIMD) These involve data-parallelism, and vector processors.

Multiple Instruction Single Data (MISD) This is not very common in practice. Redundant systems for fault tolerance are an example of MISD architecture.

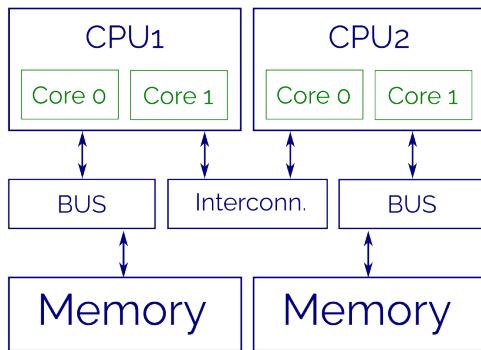
Multiple Instruction Multiple Data (MIMD) These involve task-parallelism, and are the most common type of parallel architecture. MIMD architectures can be further classified as *shared memory*, *distributed memory* or *hybrid* systems.

⁴M. Flynn, "Very high-speed computing systems", Proceedings of the IEEE **54**, 1901–1909 (1966), M. J. Flynn, "Some computer organizations and their effectiveness", IEEE Transactions on Computers **C-21**, 948–960 (1972), R. Duncan, "A survey of parallel computer architectures", Computer **23**, 5–16 (1990).

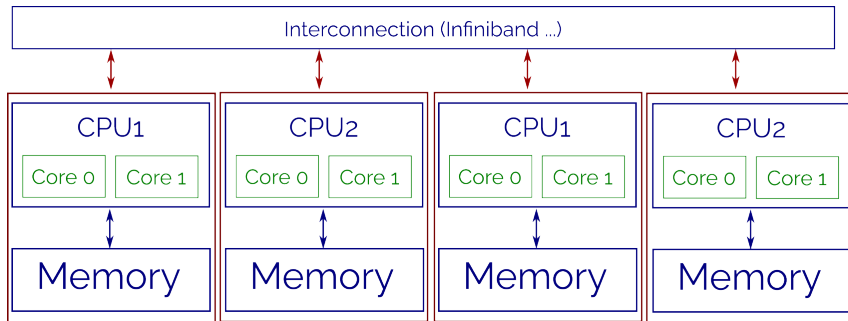
Uniform Memory Access (UMA)



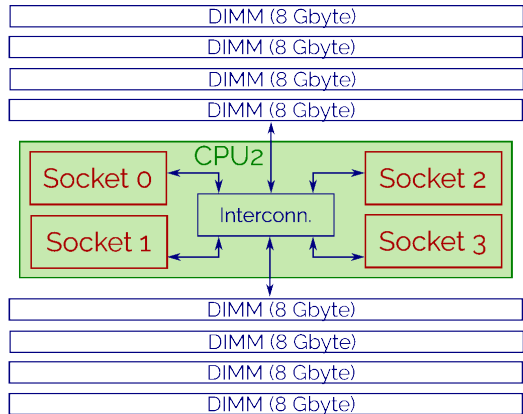
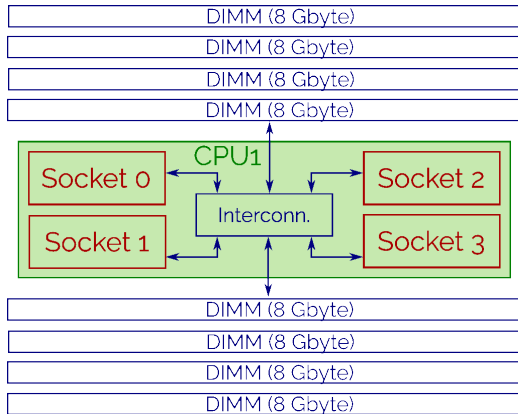
Non-Uniform Memory Access (NUMA)

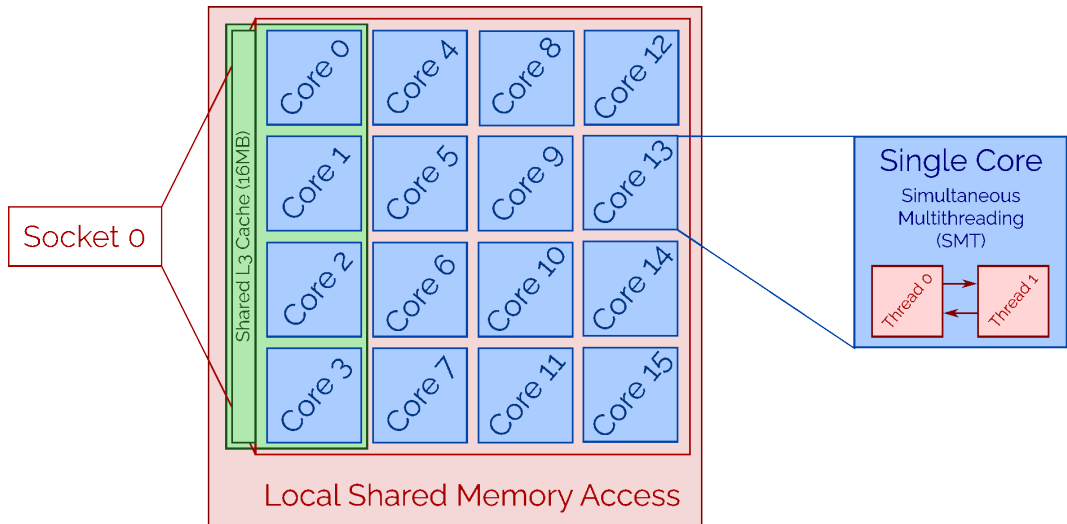


Shared-memory interconnects: bus-based, crossbar and switches.



Distributed-memory interconnects: requires a message passing API (e.g., MPI) to exchange data between the nodes.





Application Programming Interfaces





OpenMP¹ (Open Multi-Processing) is an API (Application Programming Interface) that supports multi-platform shared-memory parallel programming in C, C++, and Fortran and also GPU programming (both Nvidia and AMD).

Programming Model: Shared memory

Execution: Threads run in parallel on shared memory space

Ease of Use: Easier to implement (uses compiler directives in C/C++/Fortran)

Communication: Implicit (threads share variables by default)

Scalability: Limited by number of cores on a single machine.

Best For: Multi-core shared-memory systems.

¹www.openmp.org



MPI¹ (Message Passing Interface) is a standardized and portable message-passing system designed to allow processes to communicate with each other in a parallel computing environment — especially across distributed memory systems like clusters or supercomputers.

Programming Model: Distributed memory

Execution: Independent processes with separate memory spaces

Ease of Use: More complex (requires explicit message passing)

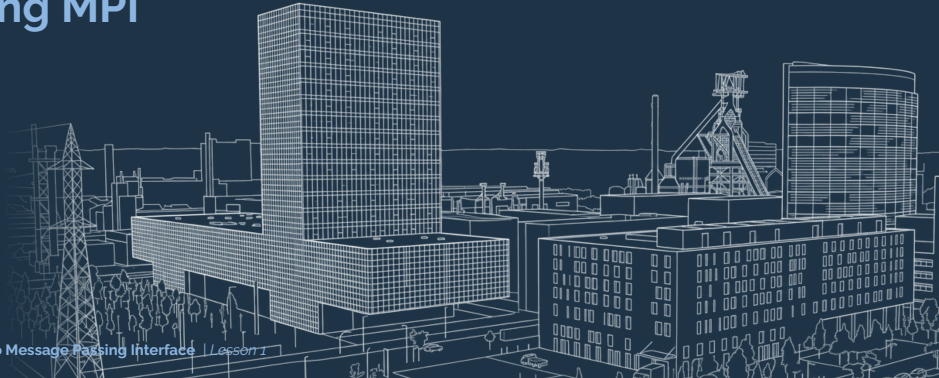
Communication: Explicit (send and receive messages between processes)

Scalability: Can scale very well across multiple nodes/machines if the computations can be uniformly distributed.

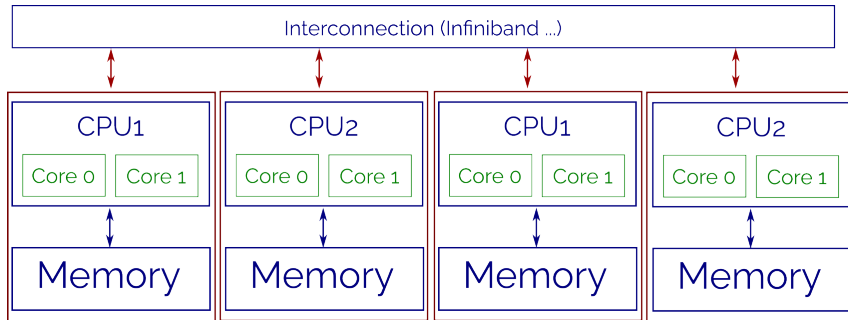
Best For: Large-scale distributed computing

¹www.mpi-forum.org, www.open-mpi.org

Introducing MPI



In the context of multiple instruction, multiple data (MIMD), distributed-memory system consists of a collection of core-memory pairs connected by a network, and the memory associated with a core is directly accessible only to that core.



Message Passing Interface (MPI) defines a library of functions that can be called from C, C ++, and Fortran programs.

```
1  #include <mpi.h>
2  #include <stdio.h>
3
4  int main(int argc, char *argv[]) {
5
6      // Initialize the MPI environment
7      MPI_Init(&argc, &argv);
8
9      // Get the number of processes
10     int num_of_ranks;
11     MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
12
13     // Get the rank of the process
14     int mpi_rank;
15     MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
16
17     // Print hello world from all MPI tasks
18     printf("Hello world from MPI process %d out of %d processes\n", mpi_rank, num_of_ranks);
19
20     // Finalize the MPI environment
21     MPI_Finalize();
22
23     return 0;
24 }
```

At the beginning of the code we include the MPI header file, which contains the declarations of all the MPI functions and constants.

```
1 #include <mpi.h>
```

After the main function the MPI environment must be initialized

```
5 int main(int argc, char *argv[]) {  
6  
7     MPI_Init(&argc, &argv);
```

where `argc` and `argv` are optional arguments that can be used (not implemented) to pass information to the MPI environment. It is also accepted to use `MPI_Init(NULL, NULL)` if no arguments are passed to the MPI environment.

At the end of the code we find

```
26     . . .  
27     MPI_Finalize();  
28  
29     return 0;  
30 }
```

that closes the MPI environment and returns control to the operating system.



Within the MPI environment we call two MPI functions defined in `mpi.h`

```
10  int num_of_ranks;  
11  MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);  
12  
13  int mpi_rank;  
14  MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
```

the first of which returns the total number of processes in the communicator `MPI_COMM_WORLD` (the default communicator that includes all the processes) and the second returns the rank of the calling process in the communicator `MPI_COMM_WORLD`.

(Notice that `&mpi_rank` and `&num_of_ranks` are the addresses of the variables `mpi_rank` and `num_of_ranks`, respectively, which are passed to the functions as pointers).

The rank is a unique identifier for each process, ranging from 0 to `num_of_ranks-1`.

The two functions

```
1  int MPI_Comm_size(  
2      MPI_Comm comm  
3      int* num_of_ranks  
4  );  
5  
6  int MPI_Comm_rank(  
7      MPI_Comm comm  
8      int* mpi_rank  
9  );
```

return an integer value, that we do not usually check (0 is success).

`MPI_Comm` is a **communicator type**, that represents a group of processes that can communicate within each other.

The default communicator is indicated through the variable `MPI_COMM_WORLD`, which includes all the processes in the MPI program.

(Communicators can be created within a program. Too advanced for this course)

Compiling, linking and executing with MPI



On a laptop machine in order to use MPI it is essential to install for example Intel MPI or OpenMPI libraries.

Latest version of the OpenMPI package can be found in www.open-mpi.org¹

After the installation it is essential to export the bin and lib directories

```
export PATH=$HOME/Software/OpenMPI-5.0.6/bin:$PATH  
export LD_LIBRARY_PATH=$HOME/Software/OpenMPI-5.0.6/lib:$LD_LIBRARY_PATH
```

On Meluxina or the ULHPC clusters the MPI libraries are usually loaded together with the toolchains

```
Meluxina: module load foss,module load intel
```

```
ULHPC (Aion/Iris): module load toolchain/foss,module load toolchain/intel
```

¹www.mpi-forum.org, www.open-mpi.org

By loading the MPI libraries, or exporting the system variables, different wrappers will be made available

```
mpicc, mpif90, mpic++, mpiifort, mpicxx ...
```

To check the characteristics of each wrapper (compiler, options, ecc.) it is sufficient to run the command

```
mpicc -show, mpif90 -show, mpic++ -show, mpicxx -show, ...
```

To compile a simple C application such as the one defined in the previous slides we run:

```
mpicc -o myapplication.exe myapplication.c
```

or for Fortran 90 and higher:

```
mpif90 -o myapplication.exe myapplication.f90
```

The option `version` of the wrappers

```
mpicc --version
```

details the compiler associated to the wrapper

```
gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0
```

To check the details of the wrapper

```
mpicc --show
```

That prints the full string of compiler and links to the MPI library

```
gcc -I/home/matteo/Software/OpenMPI-5.0.6/include  
-L/home/matteo/Software/OpenMPI-5.0.6/lib -Wl,-rpath -Wl,  
/home/matteo/Software/OpenMPI-5.0.6/lib -Wl,  
--enable-new-dtags -lmpi
```

First you export the directories of the MPI libraries

```
export PATH=$HOME/Software/OpenMPI-5.0.6/bin:$PATH  
export LD_LIBRARY_PATH=$HOME/Software/OpenMPI-5.0.6/lib:$LD_LIBRARY_PATH
```

Assuming `${num_of_ranks} = 4` we have as are result of running the application:

```
mpirun -n ${num_of_ranks} application.exe
```

The result of the execution gives:

```
Hello world from MPI process 0 out of 4 processes  
Hello world from MPI process 3 out of 4 processes  
Hello world from MPI process 1 out of 4 processes  
Hello world from MPI process 2 out of 4 processes
```

(Asynchronous writing)

To execute an application on an HPC, for example AION. A typical script `job.sh` can be written as

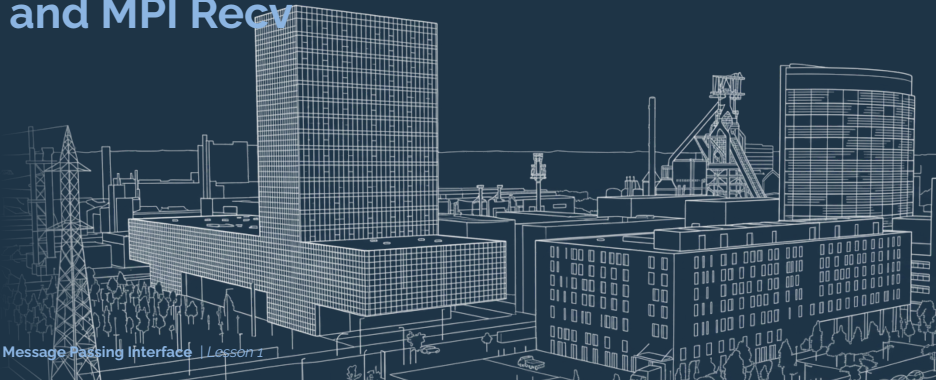
```
1  #!/bin/bash -l
2  #SBATCH -N 1
3  #SBATCH --ntasks-per-node 128 # MPI processes per node
4  #SBATCH --cpus-per-task 1      # Cores per MPI process (OpenMP)
5  #SBATCH --time=0-01:00:00
6  #SBATCH -p batch
7
8  module load toolchain/foss
9
10 export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
11
12 srun -n $SLURM_NTASKS myapplication.exe > output
```

Here we request **128 MPI tasks distributed in 1 AION node**, each task as available 1 core .

To execute we type the command

```
sbatch -J ${name_of_job} job.sh
```

MPI Send and MPI Recv



1.3.MPI_hello_world_send_recv.c

MPI Send and MPI Recv

```
1  #include <mpi.h>
2  #include <stdio.h>
3
4  int main(int argc, char *argv[]) {
5      int num_of_ranks, mpi_rank, mpi_rank_receive;
6
7      MPI_Init(&argc, &argv);
8      MPI_Comm_size(MPI_COMM_WORLD, &num_of_ranks);
9      MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
10
11     if (mpi_rank != 0){
12         MPI_Send( &mpi_rank, 1, MPI_INT, 0, mpi_rank, MPI_COMM_WORLD);
13     } else {
14         printf("Hello from process %d of %d\n", mpi_rank, num_of_ranks);
15         for (int i = 1; i < num_of_ranks; i++) {
16             MPI_Recv( &mpi_rank_receive, 1, MPI_INT, i, i, MPI_COMM_WORLD, MPI_STATUS_IGNORE)
17             ;
18             printf("Hello from process %d of %d processes\n", mpi_rank_receive, num_of_ranks)
19             ;
20         }
21     }
22     MPI_Finalize();
23     return 0;
24 }
```

```
1 int MPI_Send(const void *buffer,  
2             int buffer_size,  
3             MPI_Datatype datatype,  
4             int destination_rank,  
5             int tag,  
6             MPI_Comm comm);
```

```
1 int MPI_Recv(void *buffer,  
2             int buffer_size,  
3             MPI_Datatype datatype,  
4             int source_rank,  
5             int tag,  
6             MPI_Comm comm,  
7             MPI_Status *status);
```

buffer is the address of the first element of the data to be sent.

buffer_size is the number of elements to be sent. In our example we send only one integer, so the value is 1.

datatype is the type of the data to be sent. In our example **MPI_INT** is used to send an integer.

destination_rank is the rank of the process that will receive the data. The main rank receives the data in our example.

source_rank is the rank of the process that will receive the data.

tag is an integer that can be used to identify the message. Important to match the send and receive operations.

comm is the communicator that defines the group of processes that can communicate with each other. In our case **MPI_COMM_WORLD** is used to communicate with all the processes.

status is the communicator that defines the group of processes that can communicate with each other.

C Data Type	MPI Datatype
char	MPI_CHAR
signed char	MPI_SIGNED_CHAR
unsigned char	MPI_UNSIGNED_CHAR
short	MPI_SHORT
unsigned short	MPI_UNSIGNED_SHORT
int	MPI_INT
unsigned int	MPI_UNSIGNED
long	MPI_LONG
unsigned long	MPI_UNSIGNED_LONG
long long	MPI_LONG_LONG
unsigned long long	MPI_UNSIGNED_LONG_LONG
float	MPI_FLOAT
double	MPI_DOUBLE
long double	MPI_LONG_DOUBLE

MPI is a push first approach, meaning that **the sender pushes the data to the receiver, and continues the execution without waiting, or knowing if the receiver has received the information.**

The receiver must be ready to receive the data, otherwise the data will be lost.

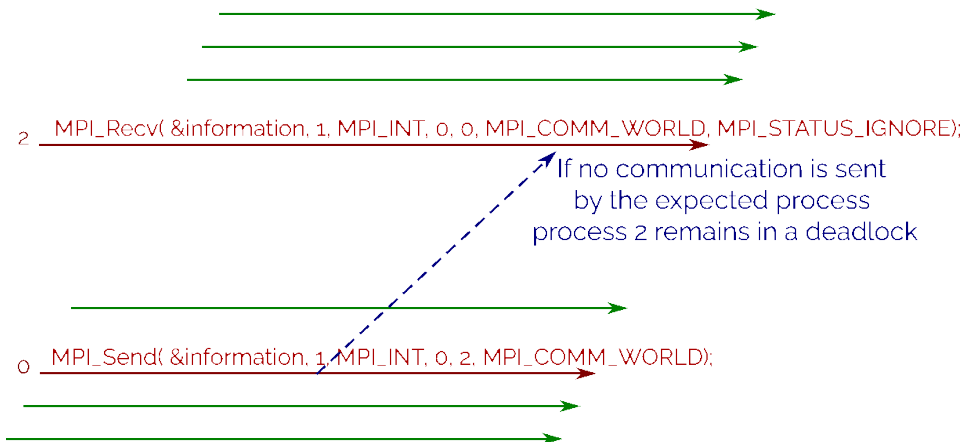
The receiver does not know: the amount of data in the message, the sender of the message, the tag of the message.

This information can be obtained by the receiver through the `MPI_Status` structure that contains at least 3 attributes

- `status.MPI_SOURCE` contains the rank of the sending process
- `status.MPI_TAG` contains the tag of the message assigned by the sender.
- `status.MPI_ERROR` contains the error code of the receive operation.

Failed Transfer Creates a Deadlock

MPI Send and MPI Recv



The communication through MPI_Send and MPI_Recv functions

```
1 int MPI_Send(sending_buffer,  
2             sending_buffer_size,  
3             sending_datatype,  
4             destination_rank,  
5             sending_tag,  
6             sending_mpi_comm);
```

```
1 int MPI_Recv(receiving_buffer,  
2             receiving_buffer_size,  
3             receiving_datatype,  
4             source_rank,  
5             receiving_tag,  
6             receiving_mpi_comm,  
7             status);
```

is successful if the following conditions are met:

- `sending_tag = receiving_tag`
- `source_rank = destination_rank`
- `receiving_mpi_comm = sending_mpi_comm`

Compatibility between sending and receiving buffers requires that the data type and size of the sending buffer is equal to that of the receiving buffer. A 'safe' communication can still happen if:

- `receiving_buffer_size ≥ sending_buffer_size`
- `receiving_datatype = sending_datatype`

Exercise: Basic numerical Integration



Let us consider a finite integral

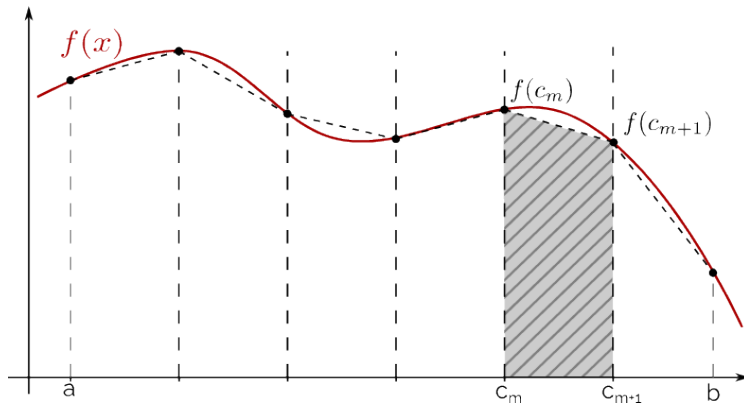
$$I = \int_a^b f(x) dx. \quad (1)$$

we can obtain an approximate numerical estimation without knowledge of the antiderivative (or primitive function) $F(x)$ of $f(x)$ by discretizing the integral space in $\mathcal{M}$ subspaces so that:

$$I = \int_a^b f(x) dx = \sum_{m=1}^{\mathcal{M}} \int_{c_m}^{c_{m+1}} f(x) dx = \sum_{m=1}^{\mathcal{M}} I_m \quad (2)$$

with $c_1 = a$ and $c_{\mathcal{M}+1} = b$ and $a \leq c_m \leq b$.

By choosing $\mathcal{M}$ as large as we want, we can reduce the subspaces of I_m , until in those intervals the function $f(x)$ behaves in a *sufficiently regular* manner.



$$I = \sum_{m=1}^{\mathcal{M}} I_m \quad I_m = (c_{m+1} - c_m) \frac{f(c_m) + f(c_{m+1})}{2} \quad (3)$$

We first define the number of intervals and the limits of the integration space

```
1  int num_of_intervals=100;
2  double a=0.0;
3  double b=1.0;
```

We set the interval lengths to a constant variable h

```
4  // Define interval length
5  if (a<b) {
6      double h = (b-a)/((double)num_of_intervals);
7  } else if ( a>b) {
8      double h = (a-b)/((double)num_of_intervals);
9  }
```

We compute the integral using the trapezoidal rule

```
10 // Compute Integral
11 double integral = my_function(&a)/2.0;
12 for (int i=1;i<num_of_intervals;i++){
13     double x=a+h*(double)(i);
14     integral+=my_function(&x);
15 }
16 integral+= my_function(&b)/2.0;
17 integral=h*integral;
```

Use as an example the serial code `1.3.SERIAL_TrapezodialRule.c`, update the function to integrate and the integration interval and parallelize the code using `MPI_send` and `MPI_recv`.

1. Integrate the function cosine in the interval $[-\pi/2, \pi/2]$.
2. Rank 0 divides the integral space $[-\pi/2, \pi/2]$ into $\mathcal{M}$ elements equal to the total number of MPI processes and computes the interval h .
3. Rank 0 communicated the value of h to all MPI processes
4. Each process (even rank 0) integrates over their subspace $[c_m, c_{m+1}]$ obtaining I_m values.
5. Collect all values from each process to the main one, (rank 0)
6. Rank 0 sums the collected values and prints the integral.